

[Experiment, Analysis, and Benchmark]

EviStateBench: Evaluating Temporal Task-State View Maintenance over Embodied Observation Streams

Jiaxiang Zhou and Yizheng Huang
Zhejiang University, China
{22421190,22421168}@zju.edu.cn

Abstract—Embodied agents increasingly produce streams of observations from perception modules, action logs, trackers, task monitors, and simulators. These streams are noisy, delayed, out of order, incomplete, and sometimes contradictory, yet downstream task execution often needs an auditable view of task state: whether an object is inside a container, whether a device is open or toggled on, whether a goal predicate currently holds, and how that answer changes under late evidence. Existing embodied benchmarks emphasize task completion, navigation, question answering, or memory recall; they do not isolate the data-management problem of maintaining temporal task-state views from imperfect observation streams.

We introduce EviStateBench, an Experiment, Analysis, and Benchmark contribution for temporal task-state view maintenance over embodied observation streams. EviStateBench derives hidden state timelines from simulator truth and task specifications, emits public StateObservation streams, provides workloads for bitemporal state, provenance, change-set, uncertainty, precondition, failure-localization, and goal queries, and evaluates predicted answers against hidden answer sets. The reported artifact contains 570 simulator-grounded episodes over 36 activities, 5,848 hard queries, 2.86M clean observations, and 2.80M hard mixed observations, with public-artifact, query-semantics, and answer-schema validation passing without errors. We also provide EviStateDB, a reference baseline engine that maintains bitemporal task-state views with support and contradict evidence. Experiments with recall-memory, temporal-log, and EviStateDB baselines show that the benchmark separates retrieval memory, temporal-log lookup, and materialized temporal-state maintenance: exact accuracy ranges from 25.2% to 75.6%, while structured, set-valued, and task-derived queries remain challenging.

Index Terms—benchmarking, temporal data management, stream processing, embodied AI, task state, provenance, uncertain data

I. INTRODUCTION

Embodied agents observe the world through imperfect streams. A robot may receive a detector output that a cabinet is open, a relation estimate that a cup is inside the cabinet, an action log that a place operation succeeded, and a later visual observation that appears to contradict the earlier relation. These observations arrive from different sources, at different times, and with different reliability. Some refer to the current world state; others arrive late and describe an earlier event.

Some are missing because an object was occluded. Some are wrong because perception failed. A task executor nevertheless needs to ask database-like questions: does the goal hold now, what did the agent know at a previous transaction time, which state changed during the episode, and which evidence supports or contradicts an answer?

Current embodied benchmarks and simulators provide important foundations, but they do not directly evaluate this maintenance problem. BEHAVIOR describes household activities with predicate-logic initial and goal conditions [1], and BEHAVIOR-1K extends the scale and simulation environment for long-horizon mobile manipulation [2]. Habitat, AI2-THOR, VirtualHome, and TEACH provide embodied environments, task programs, rearrangement, or interactive instruction-following settings [3]–[7]. OpenEQA evaluates open-vocabulary embodied question answering over episodic memory and active exploration [8]. ALFRED evaluates grounded instruction following for everyday tasks [9]. These benchmarks expose task, language, perception, and interaction challenges, but the primary scoring target is not whether a data system can maintain a temporal task-state view with valid-time semantics, transaction-time repair, provenance, and query workloads over public observation streams.

This distinction matters for data engineering. In a task such as placing a cup into a cabinet and closing the door, the relevant maintained states may include `inside(cup, cabinet)`, `open(cabinet)`, and `grasped(robot, cup)`. The query *was the cup inside the cabinet after the place action?* is not a retrieval query over similar frames. It is a temporal state query over evidence. The query *what changed between the beginning and end of the episode?* is not a single symbolic-state lookup. It is a state-diff query over a maintained view. The query *would the system have known this before the delayed observation arrived?* requires event time and arrival time to be separated. These requirements connect embodied observation streams to temporal databases, stream processing, incremental view maintenance, uncertain data, and provenance [10]–[17].

We introduce EviStateBench, a data-management benchmark for evaluating temporal task-state view maintenance over

embodied observation streams. A EviStateBench task instance provides public inputs, including task specifications, observation streams, queries, and metadata. It withholds hidden timelines and answer sets used by the evaluator. Systems under test may use any internal representation: retrieval memory, temporal logs, SQL scans, rules, materialized views, learned memory, or a hybrid. They are compared only by their predicted query answers under a shared evaluator. This public/hidden split is central: the oracle is derived from simulator truth and task specifications, not from any submitted system.

We also provide EviStateDB, a reference baseline engine. EviStateDB maintains a TemporalStateView with valid-time intervals, transaction-time revisions, support and contradict evidence, confidence, and task-derived views. It is useful for diagnosing what a temporal view-maintenance engine can do under the benchmark, but it is not the benchmark oracle. This separation lets EviStateBench evaluate EviStateDB alongside simpler baselines such as latest observation, temporal-log scan, SQL scan, and retrieval-style memory.

This paper makes four contributions.

- We formulate temporal task-state view maintenance over embodied observation streams as a data-management benchmark problem.
- We define the EviStateBench artifact boundary, observation schema, query workloads, perturbation regimes, answer schema, and evaluator semantics.
- We describe a simulator-grounded generator that produces hidden state timelines, clean and perturbed public observation streams, query sets, hidden answer sets, and validation reports.
- We provide EviStateDB as a reference baseline engine and define an experimental protocol for comparing systems under shared public inputs and hidden oracle answers.

The intended EAB contribution is therefore not a new perception model, planner, or robotics controller. It is the benchmark object: a standardized artifact, a query workload, an evaluator, a public/hidden boundary, and an analysis protocol for systems that maintain temporal task-state views. This mirrors database benchmark and EAB papers that introduce evaluation frameworks and diagnostic workloads to expose trade-offs that are hard to see in one-off experiments [18]–[24].

II. MOTIVATION AND BENCHMARK SCOPE

A. A Running Task-State Example

Consider a household task in which an agent must put a cup in a cabinet and close the cabinet. The task is naturally described by predicates over objects and time: the cup should be inside the cabinet, the cabinet should eventually be closed, and the robot should no longer be grasping the cup. A simulator or action script can provide the hidden state changes, but a deployed system observes a stream of partial evidence. A relation detector may report `inside(cup, cabinet)` with moderate confidence after the place action. A visual detector

may miss the cup after it becomes occluded. An action log may assert that the place action succeeded. A later observation may arrive after the close action and repair an earlier belief.

This example separates three notions that are often collapsed. First, *world state* is what actually holds in the simulator at a valid time. Second, *observation evidence* is what a source claims, possibly late or incorrectly. Third, *maintained task-state view* is what a system can answer after ingesting the public stream. EviStateBench evaluates the third object against hidden answers derived from the first object, using only the second object as public input.

B. What the Benchmark Evaluates

EviStateBench evaluates systems that maintain queryable temporal views of task state. A system is given a task specification, an observation stream, and a query set. It outputs predicted answers. The evaluator checks those answers against hidden answer sets. The system is not required to expose its internal state view; it may implement a log scan, a materialized view, a database table, a retrieval index, or a learned model. The only required output is the standardized predicted QueryAnswer.

The benchmark focuses on eight query families. `AS_OF_STATE` asks what would be known for a valid time at a specified transaction time. `WHY_STATE` asks which evidence supports the maintained state. `STATE_DIFF` asks which states changed between two valid-time points or intervals. `FIND_UNCERTAIN_STATES`, `CHECK_PRECONDITION`, and `FAILURE_LOCALIZATION` expose set-valued uncertainty, task-precondition, and failure-analysis views. `CHECK_STATE` and `CHECK_GOAL` provide compact point-state and goal-state probes.

C. What the Benchmark Deliberately Excludes

EviStateBench is not a robot-control benchmark. It does not score low-level manipulation success, exploration policy, path planning, or visual perception quality as primary outcomes. Those components are important, but mixing them into the main benchmark would make it difficult to attribute failures to temporal state-view maintenance. The main artifact therefore uses simulator-grounded hidden timelines and controlled public observation streams. Action-source and perception-source gaps are stated as scope limits; the main correctness claims are made over the controlled data-management workload.

This scope is intentionally narrow. A benchmark for view maintenance must make the stream, queries, hidden answers, and evaluation semantics stable. Otherwise, an accuracy difference may reflect perception quality, controller robustness, simulator physics, or task policy rather than the state-maintenance method being evaluated. EviStateBench isolates the data-management layer so that systems can compare maintenance strategies under the same public inputs.

D. Benchmark Requirements

The running example gives rise to five benchmark requirements. *R1: temporal semantics*. The benchmark must ask

about both current and historical task states, not only final task success. *R2: information availability*. It must separate valid time from arrival time so that late evidence can repair a previous state without pretending that the system knew it earlier. *R3: evidence sensitivity*. It must include support and contradict observations because task states can become uncertain or conflicting. *R4: task-derived views*. It must evaluate predicates in the context of task goals, not only isolated object attributes. *R5: reproducibility*. It must provide public inputs, hidden answers, validation reports, and a shared evaluator.

Table I positions these requirements against related benchmark families. The point is not that prior benchmarks are deficient for their own goals; rather, they target different layers of the embodied stack. EviStateBench fills the data-management layer between observation-producing embodied systems and task executors that need queryable, auditable state.

III. PROBLEM DEFINITION

A. Episodes, Task Specifications, and Hidden Timelines

An episode e is an execution instance of a task specification. A task specification defines objects, predicates, initial conditions, goal conditions, and task-level meta-data. Let $\mathcal{P}$ be the set of task-state predicates, such as *open*, *inside*, *toggled_on*, *covered*, *cooked*, *temperature*, *frozen*, and *attached*. A state atom has the form $p(\mathbf{a}) = v$, where $p \in \mathcal{P}$, $\mathbf{a}$ is a tuple of object identifiers or values, and v is a Boolean, categorical, or numeric value.

For each episode, the generator constructs a hidden state timeline

$$H_e = \langle h_1, \dots, h_m \rangle, \quad (1)$$

where each event h_i records a state atom, a value, and a valid-time interval or transition time derived from simulator truth and task specifications. Hidden timelines are not part of the public input. They are used to generate hidden answer sets.

B. StateObservation Streams

The public input stream is a sequence of StateObservations. Each observation is a tuple

$$o = (\text{obs_id}, \text{episode_id}, \text{event_time}, \text{arrival_time}, \text{source}, \text{kind}, \text{predicate}, \text{arguments}, \text{value}, \text{confidence}, \text{polarity}, \text{evidence_ref}, \text{metadata}). \quad (2)$$

The *event time* is the time to which the observation refers in the episode. The *arrival time* is the time at which the system receives the observation. The two may differ, which creates the need for transaction-time semantics. The *polarity* indicates whether the observation supports, contradicts, or corrects a state claim. The *evidence reference* links to the evidence item in the artifact, such as a frame identifier, trajectory identifier, source record, or simulator-derived annotation.

An observation stream can be clean or perturbed. Clean streams reflect simulator-derived observations with minimal injected noise. Perturbed streams inject missing observations,

temporal disorder, low confidence, and contradictions. The reported release includes a target-aware hard mixed stream that combines several issues and acts as the primary stress test.

The distinction between event time and arrival time is central. If an observation with *event_time* = 10 arrives at *arrival_time* = 40, then a system answering at transaction time 20 should not be credited for using it, while a system answering at transaction time 50 may use it to repair the maintained view. This is the benchmark analogue of bitemporal data management: valid time describes the modeled world, and transaction time describes information availability. In embodied streams, this mismatch arises naturally from delayed model inference, batch perception, communication delays, and post-hoc task-monitor updates.

C. Temporal Task-State Views

A TemporalStateView is the conceptual object being evaluated. For an episode, it is a maintained relation

$$V = (\text{state_id}, \text{episode_id}, \text{predicate}, \text{arguments}, \text{value}, \text{valid_start}, \text{valid_end}, \text{transaction_start}, \text{transaction_end}, \text{confidence}, \text{support}, \text{contradict}, \text{status}, \text{revision_history}). \quad (3)$$

Valid time records when a state is believed to hold in the world. Transaction time records when a system knew or stored that belief. Support and contradict sets record evidence used in the maintained answer. A system under test is not required to materialize V or expose it. TemporalStateView is the logical target that lets the benchmark define query semantics.

D. Queries and Answers

A query q is a public request over an episode, a predicate or goal specification, and one or more time parameters. The main query types are shown in Table II. A predicted answer contains a query identifier, an answer status, a value or change set, an optional confidence, and optional evidence. Hidden answer sets are generated from the hidden timeline and task specification. For bitemporal queries, the answer depends on both valid time and transaction time.

The answer schema is deliberately small. A QueryAnswer contains the query identifier, an answer status, a normalized value or change set when applicable, a confidence score when the system provides one, and optional evidence identifiers. The status can be *known*, *unknown*, *uncertain*, *conflict*, or *n/a*. This lets the evaluator distinguish a wrong Boolean value from a system that correctly reports that the public stream does not justify a definite answer. In the reported artifact, clean answers are mostly *known*, while hard-mixed answers include *uncertain*, *unknown*, and *conflict* states.

TABLE I
BENCHMARK POSITIONING. RELATED EMBODIED BENCHMARKS PROVIDE TASK, SIMULATION, LANGUAGE, OR MEMORY-RECALL EVALUATION, WHILE EVIStateBENCH ISOLATES TEMPORAL TASK-STATE VIEW MAINTENANCE.

Benchmark family	Primary target	Temporal queries	state	Event/arrival time	Perturbed observations	Task-derived views	Hidden oracle
BEHAVIOR	/ Household task execution	Partial		No	No	Goal predicates	Simulator truth
BEHAVIOR-1K	and simulation						
ALFRED	Instruction following	No		No	No	Task success	Demo labels
OpenEQA	Embodied question answer- ing	Episodic context		No	No	NL answers	Human/LLM scoring
Retrieval memory evals	Recall from stored observa- tions	Limited		No	Usually no	No	Dataset labels
EviStateBench	Temporal task-state maintenance	view	Yes	Yes	Yes	Yes	Simulator truth + specs

TABLE II
QUERY WORKLOADS IN EVIStateBENCH.

Query	Input	Evaluated capability
CHECK_STATE	predicate, arguments, valid time	Point state lookup over task predicates
AS_OF_STATE	predicate, valid time, transaction time	Bitemporal availability and repair
STATE_DIFF	scope, start time, end time	Change-set maintenance over an episode
CHECK_GOAL	task goal spec, valid time	Task-derived goal view correctness
WHY_STATE	predicate, time, evidence scope	Provenance and evidence support
UNCERTAIN	episode and state scope	Set-valued uncertainty tracking
PRECONDITION	task state and precondition scope	Task-derived precondition reasoning
FAILURE_LOC	goal subset and state scope	Failed-goal state localization

TABLE III
QUERY ANSWER FIELDS AND EVALUATOR ROLE.

Field	Meaning
query_id	Public query identifier used for joining predictions to hidden answers
status	known, unknown, uncertain, conflict, or n/a
value	Boolean, categorical, numeric, or structured predicate value
change_set	Added, removed, or changed states for STATE_DIFF
confidence	Optional system confidence used for calibration and MAE
evidence	Optional support/contradict observation identifiers

E. Answer Normalization and Correctness

The evaluator normalizes both hidden answers and predicted answers before scoring. Predicate names, object identifiers, and categorical values are canonicalized using the public task metadata. Boolean values are mapped to a fixed true/false representation. Numeric values are compared with predicate-specific tolerances when a tolerance is defined; otherwise exact normalized value matching is used. Change sets are represented as sets of normalized state atoms with before/after values, which prevents systems from receiving different scores

because of output ordering.

Correctness is not only value equality. For CHECK_STATE, a prediction is exactly correct when the status and value match the hidden answer, or when both answers mark the state as not applicable. For AS_OF_STATE, the evaluator applies the same value rule after filtering evidence by transaction time. This is the critical check against hindsight leakage: an answer can be right for the final world state but wrong for the information that was available at the requested transaction time. For STATE_DIFF, exact match requires the full change set to match, while diff F1 gives partial credit for overlapping changed atoms. For CHECK_GOAL, exact match checks the task-level status and goal truth value, while goal-predicate F1 measures the predicate-level explanation.

Status semantics are scored explicitly because they are central to imperfect streams. A system should not be forced to guess a Boolean value when the public evidence is insufficient. Conversely, a system should not hide behind unknown for every difficult query. The evaluator therefore reports coverage and status accuracy alongside value metrics. A high-value but low-status system is useful when downstream components can tolerate overconfident answers; a lower-coverage system with accurate uncertainty may be preferable for safety-critical task monitoring. EviStateBench exposes this trade-off rather than collapsing it into a single scalar.

Evidence fields are optional in the core score, but they are retained for audit and why-style workloads. When a system emits support or contradict observation identifiers, the evaluator can measure whether the cited evidence overlaps with oracle evidence. The reported main score does not require evidence citation so that simple baselines remain comparable. The artifact nevertheless keeps evidence references in the public stream because provenance is part of the benchmark design.

IV. EVIStateBENCH BENCHMARK DESIGN

A. Benchmark Package

EviStateBench is a benchmark package consisting of public artifacts, hidden evaluator artifacts, and a shared evaluation protocol. The public artifacts contain task specifications, observation streams, query sets, metadata, manifests, validation reports, and build commands. The hidden evaluator artifacts

contain the hidden state timelines and answer sets. Systems under test can read only the public package. Their predictions are scored by the evaluator, which reads both the predictions and hidden answers.

EviStateDB is evaluated by EviStateBench; it does not generate the ground truth used to judge other systems. The oracle is constructed from simulator truth and task specifications. This distinction also makes the benchmark representation-agnostic: a retrieval-based system, a SQL-based system, a rule engine, and EviStateDB can all submit the same QueryAnswer schema.

The package is organized so that benchmark users can run three roles independently. A *benchmark builder* can regenerate public and hidden artifacts from the pipeline manifests. A *system developer* can download the public package, implement an answerer, and emit predictions. An *evaluator* can score predictions against hidden answers. This separation is important for artifact review and public leaderboards: the system developer does not need access to any hidden timeline files to test parsing, run baselines, or validate prediction format.

B. Artifact Boundary and Leakage Control

The public artifact exposes the information that a deployed state-maintenance system could plausibly observe: task descriptions, state observations, query specifications, and metadata needed to parse them. It does not expose hidden timeline events, answer sets, or perturbation labels that would reveal the target answer. The evaluator can be run after a system writes predictions. This design follows the common benchmark pattern of public inputs and hidden labels, but applies it to temporal state-view maintenance rather than classification or retrieval.

The public-artifact validation step checks that required manifests exist, query identifiers are well formed, observation streams are readable, task specifications align with query metadata, and answer leakage is absent from public files. In the reported artifact, validation reports 570 task specifications, 5,848 query rows, one primary hard mixed public stream, and 0 validation errors.

The artifact boundary also defines what counts as a fair comparison. All systems receive the same public observation streams in the same order. They receive the same query set and task specifications. They may use external software libraries, database engines, or learned components, but they may not read hidden answer files, hidden timeline files, or perturbation annotations that reveal which observations were injected. This rule makes the benchmark compatible with both classical data-management systems and neural memory systems while keeping the evaluation target fixed.

C. Workload Axes

EviStateBench varies several workload axes simultaneously. The main axes are query family, predicate family, episode horizon, stream regime, temporal position, and evidence status. Query family determines the answer shape. Predicate family determines whether the system is maintaining unary states,

binary relations, numeric states, contact configurations, or material states. Episode horizon controls the length of the observation stream and the number of opportunities for stale or repaired state. Stream regime controls whether the public evidence is clean or perturbed. Temporal position distinguishes initial, transition-near, and final queries. Evidence status captures whether the public stream offers support, contradict evidence, both, or neither.

These axes are not merely descriptive metadata. They determine how results should be read. A system that performs well on clean final-state checks may still fail on transition-near as-of queries. A system that handles Boolean unary predicates may not handle numeric temperature states or relation changes. A system that answers goals correctly on short episodes may degrade on long episodes because stale evidence accumulates. The evaluation therefore reports overall metrics together with query-family and stream-regime breakdowns.

D. Perturbation Regimes

The perturbation regimes are chosen to correspond to data-management failures rather than to a single sensor model. Clean streams test basic state-view construction. Missing observations test unknown and stale-state handling. Temporal disorder tests valid-time and transaction-time repair. Conflict tests confidence and provenance tracking. The reported hard mixed stream combines several issues and is the main stress condition because embodied observation streams rarely contain a single isolated issue.

E. Data Model and Query Semantics

The important semantic choice is that observations are evidence, not final state. A state answer is produced by interpreting evidence over time. For `CHECK_STATE`, the evaluator checks the value of a target state at a valid time. For `AS_OF_STATE`, the evaluator additionally constrains information availability by transaction time. For `STATE_DIFF`, the answer is a change set over a scope. For `CHECK_GOAL`, the answer is task-derived and can require multiple predicates. The same answer schema also supports provenance, uncertainty-set, precondition, and failure-localization workloads.

F. Online and Batch Evaluation Modes

The benchmark supports both online and batch evaluation modes. In online mode, a system ingests observations in arrival-time order and must answer queries at their declared transaction times. This mode best matches task monitors that operate during execution. In batch mode, a system may preprocess the full public stream but must still respect transaction-time constraints for `AS_OF_STATE` queries. Batch mode is useful for comparing database implementations and retrieval systems whose natural interface is offline.

Both modes are valid, but they answer different performance questions. Online mode emphasizes update cost, state repair, and incremental view maintenance. Batch mode emphasizes query semantics and representation quality. The reported runs identify which mode each baseline uses. When both are

available for a system, EviStateBench reports preprocessing time separately from query time so that an offline scan is not confused with an online maintained view.

G. Evaluation Metrics

EviStateBench reports both answer-level and task-view metrics. Exact accuracy checks whether the predicted structured answer matches the hidden answer for a query. Value accuracy ignores status mistakes when the value field is applicable. Status accuracy checks whether the system distinguishes `known`, `unknown`, `uncertain`, and `conflict`. `STATE_DIFF` uses both exact change-set match and mean F1 over changed state atoms. `CHECK_GOAL` reports satisfied accuracy and goal-predicate F1 so that a system can receive partial credit when it identifies some but not all satisfied or violated goal predicates.

These metrics are intentionally diagnostic. A single global score would hide the differences between state-state errors, transaction-time errors, status-calibration errors, and change-set errors. The evaluation therefore reports overall metrics, query-family breakdowns, and perturbation-regime breakdowns. For systems that emit confidence, EviStateBench also reports confidence mean absolute error against hidden answer confidence where defined.

V. BENCHMARK GENERATION AND ARTIFACTS

A. Generation Pipeline

The generation pipeline begins with task specifications and simulator-grounded episode sources. It extracts semantic state changes from simulator truth, normalizes them into hidden timeline events, produces clean `StateObservation` streams, injects perturbations to produce stress streams, generates query sets, derives hidden answer sets, and builds a sanitized public package. Each step writes a manifest or report so that the benchmark can be rebuilt and audited.

The hidden timeline is the anchor of the artifact. It records the task-state transitions from which both observations and answers are derived. Clean observations are public evidence records generated from the same timeline. Perturbed observations are public stress streams that preserve the task structure while introducing the stream defects that a maintenance system must handle. Query generation then samples state checks, bitemporal as-of probes, full-episode diffs, and task-goal checks.

B. Query Generation

The query generator is tied to hidden timeline events and task goal specifications. It first constructs a candidate query pool from target state changes and task goals. Candidate probes include `CHECK_STATE` queries at target valid times, paired `AS_OF_STATE` queries whose transaction times fall before and after the relevant observation window, full-episode `STATE_DIFF` queries, and initial/final `CHECK_GOAL` queries. The transaction-time pair tests whether systems respect information availability rather than answering every historical query from final hindsight.

The reported hard split then filters this candidate pool to form the public workload in Table V. It keeps the temporal and provenance-heavy probes, keeps one episode-level `STATE_DIFF`, `FIND_UNCERTAIN_STATES`, `CHECK_PRECONDITION`, and `FAILURE_LOCALIZATION` query per episode, and down-samples easy point-state and goal probes to 120 `CHECK_STATE` and 120 `CHECK_GOAL` queries. This two-stage design explains why the final public query counts are smaller than the full candidate pool while still preserving task-linked temporal, set-valued, and failure-analysis workloads.

This generation strategy keeps the workload close to task semantics. The benchmark is not sampling arbitrary predicate/time combinations. It asks about states that matter to task progress: target object states, transitions, and goals. It also ensures that every query has a hidden answer derived from the same timeline used to produce public observations.

The generator also records why each query exists. For example, a `CHECK_STATE` query can be linked to the hidden event that changed the target atom, and a `CHECK_GOAL` query can be linked to the goal predicates in the task specification. These links are hidden from systems under test but useful for analysis. They let the evaluator group errors by transition type, goal predicate, or perturbation source without leaking that information into the public package.

C. Perturbation Injection

Perturbations are injected after the clean stream is generated. Missing-observation perturbations remove evidence near selected state transitions. Temporal-disorder perturbations shift arrival times or reorder observations while preserving event-time references. Conflict perturbations introduce observations whose polarity or value contradicts the hidden state. Low-confidence perturbations lower confidence without necessarily changing the value. The hard mixed stream combines these effects and is used as the main stress stream in the reported public artifact.

The perturbation design is not meant to mimic every real sensor failure. Its purpose is to create controlled failure modes that correspond to data-management challenges. Missing evidence tests whether a system can represent unknown states. Temporal disorder tests bitemporal repair. Conflict tests evidence handling and status semantics. Mixed streams test whether systems remain robust when these issues co-occur.

Perturbations are applied with deterministic seeds and written to manifests. The public package does not reveal which observations were perturbed, but the hidden analysis package records perturbation provenance. This design allows two levels of reporting. The main paper reports public-regime results, such as clean versus hard mixed. A diagnostic analysis can additionally explain whether a failure was caused by missing evidence, temporal disorder, or contradiction. Keeping perturbation provenance hidden avoids turning the task into a label-reading exercise while still supporting benchmark debugging.

TABLE IV
REPORTED EVIStateBENCH ARTIFACT PROFILE.

Item	Count or status
Episodes	570
Activities	36
Task families	4
Target predicates	11
Task specifications	570
Clean observations	2,856,815
Hard mixed observations	2,797,127
Queries	5,848
Primary public stream	hard_mixed
Validation errors	0
Public validation status	PASS
Query semantics audit	PASS
Answer schema validation	PASS

TABLE V
REPORTED QUERY DISTRIBUTION IN THE MAIN ARTIFACT.

Query family	Count
AS_OF_STATE	1,772
WHY_STATE	1,556
STATE_DIFF	570
FIND_UNCERTAIN_STATES	570
CHECK_PRECONDITION	570
FAILURE_LOCALIZATION	570
CHECK_STATE	120
CHECK_GOAL	120
Total	5,848

D. Current Main Artifact

Table IV reports the artifact used in this paper, `public_final_570_hard_v1`. It contains 570 simulator-grounded episodes across 36 BEHAVIOR/OmniGibson activities, with 5,848 hard queries. The clean stream contains 2,856,815 observations; the hard mixed stream contains 2,797,127 observations. Public-artifact validation, query-semantics audit, and answer-schema validation all report PASS. These counts mark the artifact as a controlled simulator-grounded benchmark for temporal task-state maintenance, not as a claim of full real-world robot deployment coverage.

The query distribution emphasizes temporal and provenance-heavy state maintenance: 1,772 `AS_OF_STATE` queries and 1,556 `WHY_STATE` queries form the largest workload. The artifact also includes 570 `STATE_DIFF`, 570 `FIND_UNCERTAIN_STATES`, 570 `CHECK_PRECONDITION`, 570 `FAILURE_LOCALIZATION`, 120 `CHECK_STATE`, and 120 `CHECK_GOAL` queries. The target predicates cover object unary state, material state, containment, and contact configuration, with 11 target predicates including `open`, `covered`, `inside`, `cooked`, `toggled_on`, `temperature`, `attached`, `frozen`, `max_temperature`, `hot`, and `on_fire`.

E. Validation and Known Limits

The public package is validated before evaluation. The release validation verifies 570 task specifications, 5,848 query rows, and the primary hard mixed observation stream. It reports no validation errors. The artifact also records horizon splits for interpretation: 540 short controlled episodes and 30 long state-chain episodes.

Two limits are deliberately kept visible. First, the main artifact isolates temporal view maintenance from low-level robot control. Second, the main artifact uses structured simulator-derived observations rather than raw perception outputs as its correctness foundation. These choices make the benchmark reproducible and diagnostically clean. Low-level rollout and perception-derived streams are outside the main score; the correctness claim is state-view maintenance given public observation streams.

The validation status should be read in this sense. The artifact passes public validation and is suitable for evaluating the target benchmark problem, while the release notes keep scope limits explicit. ICDE EAB papers are evaluated not only by whether the artifact exists, but also by whether its boundary is clear enough for other researchers to reproduce and interpret results.

F. Reproducibility Knobs

The artifact identifier encodes the release lineage and hard-split role. The reported artifact, `public_final_570_hard_v1`, is therefore not an anonymous directory name; it is a compact description of the benchmark package used by the reported experiments. Manifests record task specifications, stream files, query sets, answer sets, and validation reports.

The generator writes manifests for task specifications, streams, queries, validation reports, and profile summaries. A reproduction run verifies that these manifests agree with the paper tables before running baselines. This is especially important for an EAB benchmark, because small inconsistencies in query counts or stream files can invalidate comparisons even when each individual system runs successfully. The artifact package includes `validate_release.py`, `profile_report.py`, and `build_hard_split.py` for release validation, profile regeneration, and hard-split construction; the baseline and figure packages include corresponding evaluation and plotting scripts for regenerating the reported results.

VI. REFERENCE ENGINE AND EXPERIMENTAL PROTOCOL

A. EviStateDB as a Reference Baseline

EviStateDB is a reference baseline engine for EviStateBench. It ingests StateObservation streams and maintains a TemporalStateView. For each state atom, it records valid-time intervals, transaction-time revisions, confidence, support evidence, contradict evidence, status, and revision history. It also derives task-facing views such as goal views, precondition views, state-diff views, uncertain-state views, and failure-analysis views when the query workload requires them.

TABLE VI
REFERENCE INGESTION IN EVIStatedB.

Require: StateObservation o , maintained view V

- 1: Locate candidate state atoms matching $o.predicate$ and $o.arguments$
- 2: **if** o supports an active state interval **then**
- 3: Update confidence and add o to support evidence
- 4: **else if** o contradicts an active state interval **then**
- 5: Add o to contradict evidence
- 6: Revise confidence and, if needed, close or split the valid interval
- 7: **else**
- 8: Insert a new state interval starting at $o.event_time$
- 9: **end if**
- 10: **if** $o.arrival_time$ is later than $o.event_time$ **then**
- 11: Record a transaction-time revision
- 12: **end if**
- 13: Refresh indexes and task-derived views affected by the state atom

EviStateDB is intentionally described as a baseline, not as the main benchmark contribution. The benchmark is designed to evaluate any system that can read the public artifacts and output QueryAnswers. EviStateDB supplies one strong, interpretable data-management baseline that demonstrates how bitemporal repair and evidence tracking can be implemented.

EviStateDB stores an observation log and materializes state intervals indexed by episode, predicate, arguments, valid time, and transaction time. Its task-derived views are computed from the same maintained state intervals rather than from hidden answers. This makes EviStateDB useful as a reference point for systems that wish to trade update cost for query speed. It also gives the benchmark a concrete implementation path for bitemporal repair, without forcing all systems to use the same representation.

B. Maintenance Sketch

Table VI sketches the reference ingestion logic. The actual implementation can vary, but the pseudocode exposes the operations that the benchmark is meant to stress: identifying relevant state atoms, deciding whether an observation supports or contradicts an existing view, revising valid-time intervals when late evidence arrives, recording transaction-time history, and retaining evidence for audit queries.

C. Baselines and Fair-Comparison Rules

The evaluation compares EviStateDB with two transparent baselines. *Recall Memory* retrieves related observations and answers from retrieved evidence without maintaining an explicit temporal task-state view. *Temporal Log* keeps the observation log and evaluates temporal queries over the log, but does not materialize the full set of task-derived views maintained by EviStateDB. These baselines are intentionally simple enough to audit; their role is to reveal whether the benchmark distinguishes retrieval-style memory, log-based temporal lookup, and explicit temporal state-view maintenance.

All systems must use the same public task specifications, observation streams, query sets, and metadata. They must not read hidden timelines, hidden answer sets, or private perturbation labels. All systems submit predictions in the same QueryAnswer format. A single evaluator computes exact

accuracy, status accuracy, value accuracy, change-set F1, goal-predicate F1, coverage, and efficiency metrics. This shared protocol is what turns EviStateBench from a data collection into an EAB benchmark.

D. Reporting Protocol

Each run report records the public artifact identifier, stream name, system version, hardware, random seed where applicable, preprocessing time, update time, query time, and output coverage. Accuracy and efficiency are reported separately. A system that answers accurately by scanning the full observation stream for every query is different from a system that maintains a materialized view online; both are valid systems under EviStateBench, but the trade-off is part of the EAB analysis.

The reported runs use the same reporting template for all baselines. This matters because benchmark papers often fail when each method is evaluated under slightly different assumptions. EviStateBench fixes the stream, queries, hidden answers, and metrics; the system implementation is the only moving part.

VII. EVALUATION AND ANALYSIS

A. Artifact Characterization

The reported artifact supports the first EAB claim: EviStateBench is a concrete, validated benchmark package rather than only a proposal. Fig. 1 summarizes the released workload. The artifact contains 570 simulator-grounded episodes, 36 activities, four task-state families, 11 target predicates, 5,848 hard queries, 2.86M clean observations, and 2.80M hard mixed observations. The query set covers point state checks, bitemporal as-of probes, provenance queries, full-episode diffs, uncertainty sets, preconditions, failure localization, and task-goal checks. The target predicates cover object unary state, material state, containment, and contact configuration. This diversity is important because a benchmark limited to one predicate family would be easy to overfit with a specialized rule.

B. Hard-Split Difficulty

Fig. 2 explains why the hard split is more than a larger file. Among 14,905 target simulator-state observations, only 2,512 remain as direct target evidence, 10,282 are downgraded to lower-confidence evidence, and 2,111 are dropped. The overall stream also contains 87,571 dropped observations, 144,011 low-confidence observations, and 27,883 injected conflicts. This perturbation is target-aware: it stresses evidence near the states that queries ask about, while leaving most background observations intact.

The hard split is therefore a controlled stress test for data-management behavior. It is not an empirical model of every real sensor failure. Its purpose is to create missing, low-confidence, delayed, and conflicting evidence regimes in which systems must decide whether a state is known, unknown, uncertain, or conflicting. The confidence and predicate panels in Fig. 2 show that the resulting workload is not binary.

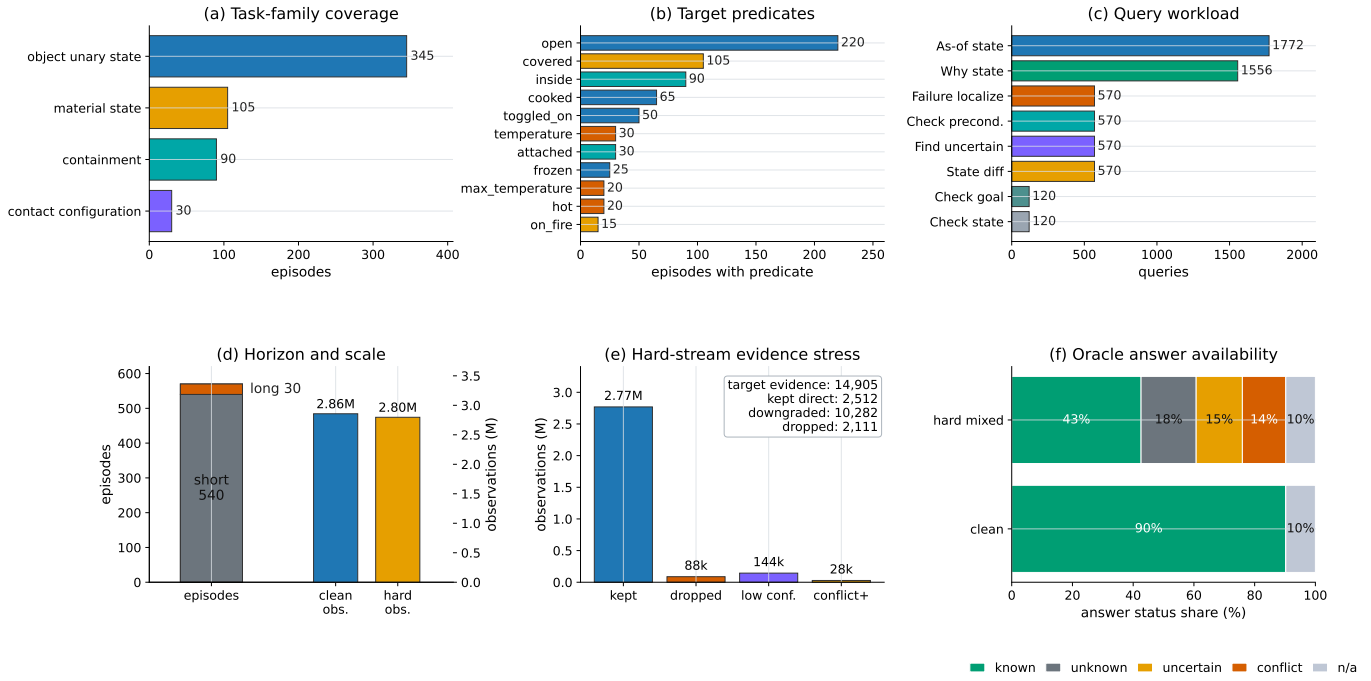


Fig. 1. Benchmark artifact profile. EviStateBench covers 570 simulator-grounded episodes over 36 activities, four task-state families, and 11 target predicates. The hard mixed stream stresses target evidence and shifts answer availability from mostly known clean answers to a mixture of known, unknown, uncertain, conflicting, and not-applicable answers.

Different predicates exhibit different answer mixtures, and temporal/provenance queries include high-confidence, low-confidence, missing, and conflicting cases.

C. Long State Chains

Most episodes in the reported artifact are short controlled transitions, but the release also includes a 30-episode long state-chain subset over cooking and freezing activities. The cooking chains move from an actuator state to temperature and maximum-temperature changes, and then to hot and cooked states. The freezing chain moves from opening the refrigerator to temperature change and then to frozen state. This subset is not large enough to claim broad long-horizon BEHAVIOR coverage, but it demonstrates that the benchmark can express multi-stage temporal state maintenance beyond a single target flip.

Fig. 3 makes this diagnostic role explicit. The long subset is small and is not a separate robotics claim; it shows that the artifact can express chained state consequences and convert them into temporal availability cases under hard mixed evidence.

D. Baseline Results

Fig. 4 compares three reference baselines on the final hard split. Recall memory answers from retrieved or recent evidence without maintaining a temporal state view. Temporal log keeps an explicit event log and evaluates temporal queries over it. EviStateDB maintains a bitemporal task-state view with support and contradict evidence.

The overall exact accuracies are 25.2% for Recall memory, 69.0% for Temporal log, and 75.6% for EviStateDB. These results support the benchmark’s diagnostic goal: the workload separates retrieval-style memory, append-only temporal lookup, and materialized temporal-state views. The largest gap is between retrieval-style memory and the two temporal baselines; the additional gain from Temporal log to EviStateDB is smaller and concentrates in structured and task-derived workloads. Temporal log and EviStateDB reach 98% exact accuracy on AS_OF_STATE and WHY_STATE, but they still struggle on structured workloads. EviStateDB reaches only 39% exact accuracy on STATE_DIFF and 10% on FIND_UNCERTAIN_STATES, although its partial structured metrics are higher: 42.4% Diff F1, 63.8% Goal F1, 72.6% State-set F1, and 92.5% Why-support F1.

The runtime panel reports per-query prediction time, not full end-to-end indexing or stream-loading time. It shows a different stress point: most point-state queries are inexpensive after indexing, while FIND_UNCERTAIN_STATES dominates p95 query time for all methods. This is useful benchmark information because it separates semantic difficulty from query-time pressure.

E. Failure and Difficulty Diagnostics

The baseline diagnostics separate two different messages. First, answer status matters. Recall memory fails especially on unknown, uncertain, and conflict answers, while Temporal log and EviStateDB recover most unknown and uncertain statuses. Second, the high predicate-level accuracy for Temporal log and

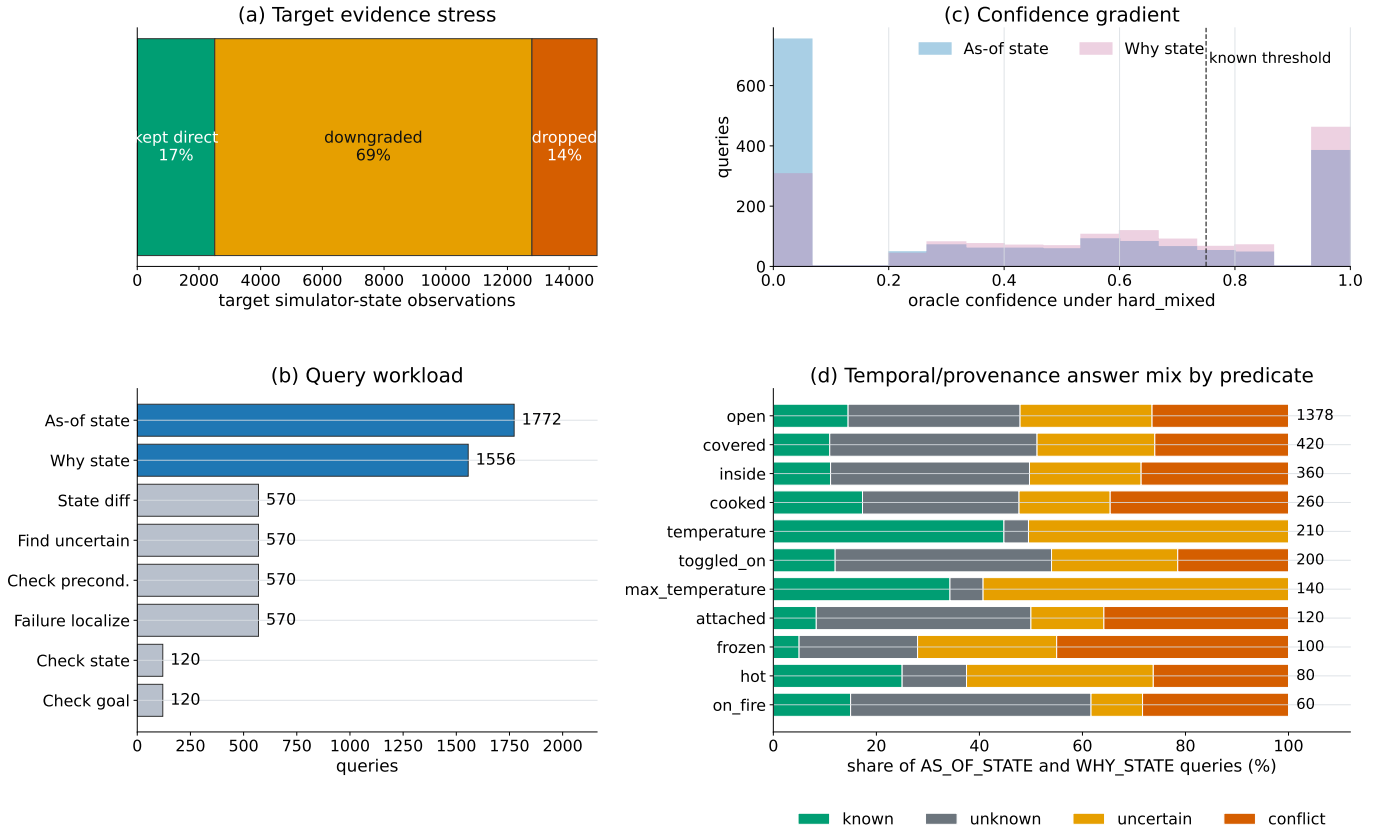


Fig. 2. Hard-split mechanism and query difficulty profile. The hard stream keeps only a minority of target simulator-state observations as direct evidence, while downgrading or dropping the rest. The resulting workload emphasizes temporal and provenance queries and produces graded known, unknown, uncertain, and conflicting answer conditions across target predicates.

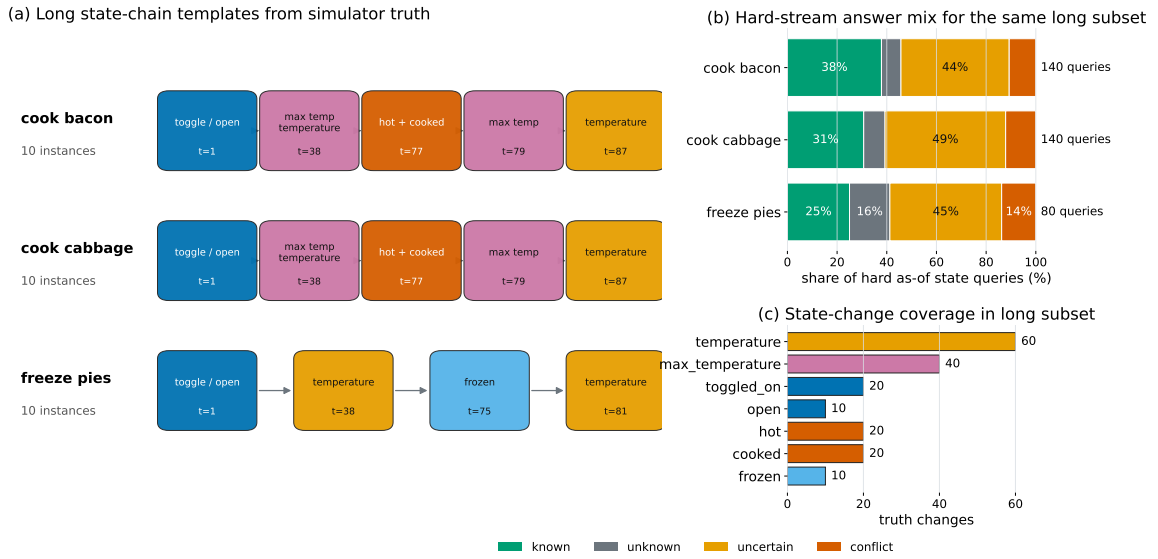


Fig. 3. Aggregated long state-chain templates. The 30-episode long subset adds multi-stage temporal maintenance cases, such as actuator, temperature, hot, cooked, and frozen state chains, to the larger short-transition benchmark.

EviStateDB on AS_OF_STATE and WHY_STATE should not be overstated. It means that explicit event-time and transaction-time maintenance is effective for these two query families. It

does not mean that all of EviStateBench is easy, because the residual errors in Fig. 4 concentrate in set-valued uncertainty, state diffs, preconditions, failure localization, and goal views.

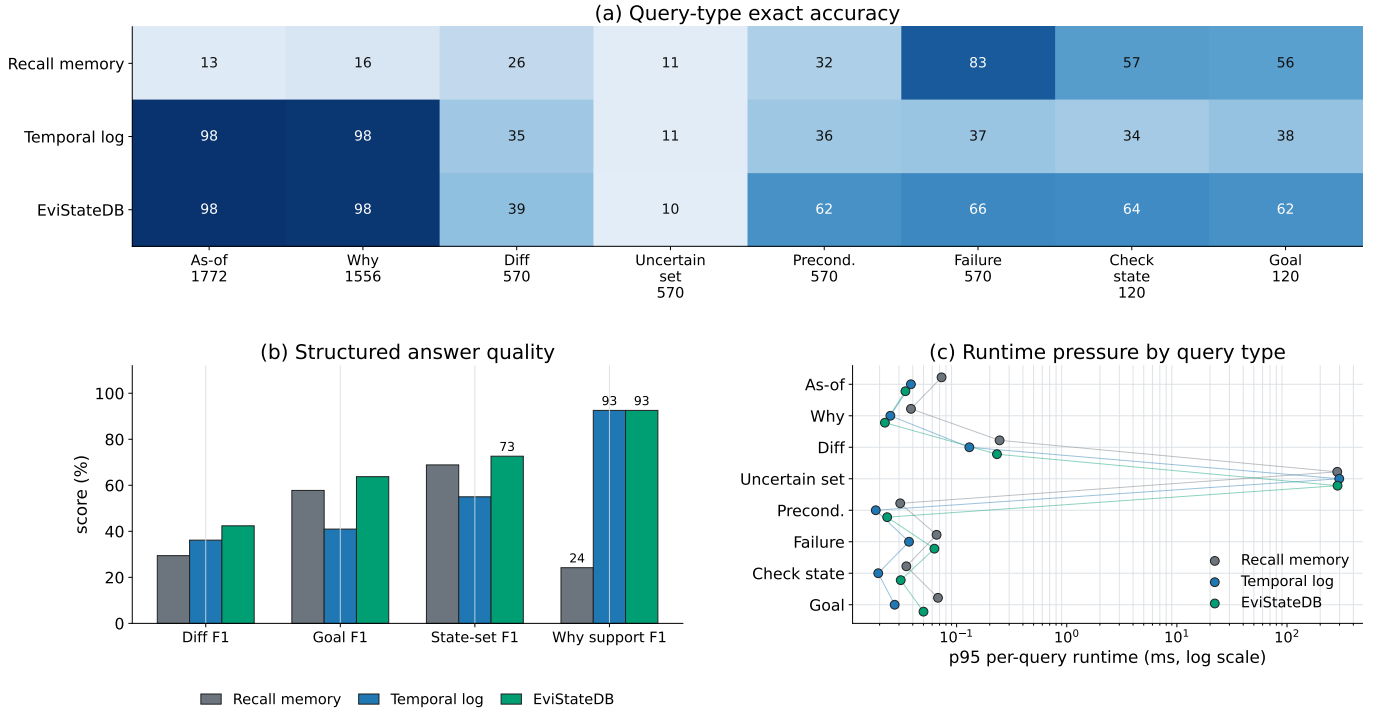


Fig. 4. Baseline results on the final hard split. The upper heatmap reports query-type exact accuracy, the lower-left panel reports structured answer metrics, and the lower-right panel reports p95 per-query runtime from prediction outputs. All methods cover the full 5,848-query workload.

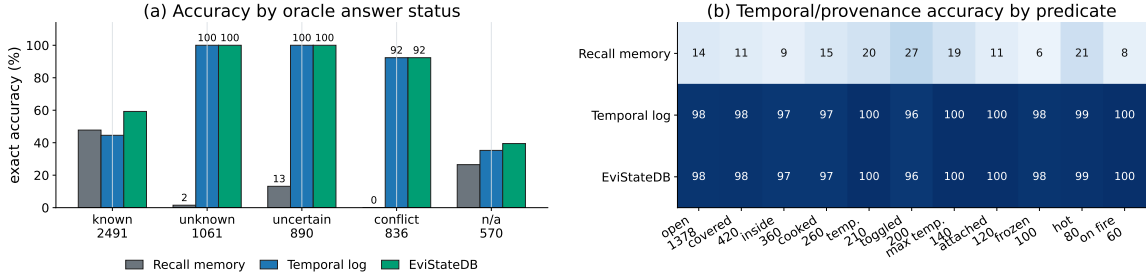


Fig. 5. Difficulty analysis by answer status and predicate. Recall memory degrades across answer statuses and target predicates, while explicit temporal-state baselines largely solve AS_OF_STATE and WHY_STATE.

Fig. 5 explains where residual difficulty comes from. Retrieval fails broadly, while Temporal log and EviStateDB largely recover temporal and provenance answers. The remaining difficulty is therefore concentrated in answer shapes that require sets, diffs, preconditions, failure localization, and goal aggregation.

The interpretation should avoid two common mistakes. It should not treat EviStateDB as the oracle merely because it is the reference engine. It should also not claim that a lower score means a method is bad for all embodied AI tasks. A retrieval memory that performs poorly on state diffs may still be useful for open-ended question answering. The EAB claim is narrower and more useful: EviStateBench identifies which systems are good at temporal task-state view maintenance under a shared, reproducible workload.

VIII. LESSONS, LIMITATIONS, AND ARTIFACT AVAILABILITY

A. Lessons from the Benchmark Design

EviStateBench exposes five practical lessons. First, latest-state memory is not a sufficient abstraction because late evidence can revise as-of answers and contradict evidence can make a state uncertain or conflicting. Second, retrieval and view maintenance answer different questions: retrieval can surface evidence, but task execution often needs a maintained predicate value, change set, or goal view. Third, provenance should be first-class because support and contradict evidence are needed for why queries, failure localization, and trust calibration. Fourth, goal views are not just convenience layers; they can fail even when individual predicates are partly correct. Fifth, validation reports are benchmark artifacts, since mal-

formed query metadata or leaked answer fields can invalidate comparisons as much as model errors.

B. Limitations

EviStateBench makes a controlled isolation choice. The main artifact is simulator-grounded and structured; it does not claim to solve raw perception or low-level robot control. This is a limitation, but also a design requirement for a reproducible data-management benchmark, because folding perception and control into the main score too early would make failures hard to attribute. The reported baseline set is also intentionally conservative: neural memory, LLM-based agents, and full embodied pipelines fit the benchmark only when their public inputs, inference budgets, and output schemas can be made comparable.

C. Artifact Availability

The benchmark release provides public task specifications, observation streams, query sets, manifests, validation reports, build commands, baseline implementations, prediction schemas, and evaluator scripts. Hidden timelines and answer sets are used for evaluation. The minimal reproduction path builds or validates public artifacts, runs each baseline on the hard mixed stream, validates prediction schema, evaluates predictions, and regenerates the reported tables and figures.

IX. RELATED WORK

A. Embodied Task and Simulation Benchmarks

BEHAVIOR introduced a benchmark of everyday household activities with predicate-logic activity specifications and simulation-based evaluation [1]. BEHAVIOR-1K scales this line of work to 1,000 activities and introduces OmniGibson as a simulation environment for realistic physics and rendering [2]. Habitat and Habitat 2.0 support embodied navigation and rearrangement evaluation [3], [4]; AI2-THOR provides an interactive 3D environment for visual AI [5]; VirtualHome represents household activities as executable programs [6]; and TEACH studies embodied agents that interact through dialogue [7]. EviStateBench uses the same general idea that task specifications and simulator truth can ground evaluation, but it changes the target from embodied task success to temporal task-state view maintenance.

ALFRED evaluates instruction following [9], and OpenEQA evaluates embodied question answering over episodic memory and active exploration [8]. These benchmarks ask whether an agent can act or answer language questions. EviStateBench asks whether a system can maintain and query a temporal state view from imperfect evidence, without prescribing the system’s internal memory architecture.

B. Temporal, Stream, and Provenance Data Management

Temporal databases separate the time when a fact is valid in the modeled world from the time when a database records or knows the fact [10], [11]. Stream systems study continuous processing over arriving data [12], [25]. Incremental view maintenance studies how derived database state can be updated

after base changes [13], [14]. Provenance and uncertain-data systems address evidence and noisy facts [15]–[17], [26]. EviStateBench brings these ideas into embodied task-state maintenance, where event-time and arrival-time mismatch, missing observations, conflicting evidence, and task-derived goal views appear together.

C. Experiment, Analysis, and Benchmark Papers

Database benchmark and EAB papers often contribute workload models, artifact rules, and diagnostic analyses rather than only a new algorithm. LDBC standardizes graph workloads [18], [19]; TSM-Bench benchmarks time-series systems [20]; and FinBench defines a financial graph transaction workload [21]. Recent EAB papers similarly contribute evaluation frameworks and fine-grained diagnostics, including SQLMorph [22], BEACON [23], and plan-based AQP analysis [24]. EviStateBench follows this pattern: the contribution is the benchmark artifact, workload, evaluator, and analysis of where baseline strategies succeed or fail.

X. CONCLUSION

EviStateBench is a data-management benchmark for temporal task-state view maintenance over embodied observation streams. It separates public observation streams and queries from hidden timelines and answers, evaluates systems through standardized QueryAnswers, and makes event time, arrival time, evidence, perturbations, and task-derived state views first-class benchmark concerns. EviStateDB provides a reference baseline engine, but the benchmark oracle comes from simulator truth and task specifications. The reported artifact and experiments show that temporal maintenance strategies outperform retrieval-style memory on this workload, while structured, set-valued, and task-derived queries remain open stress points for submitted systems.

AI-GENERATED CONTENT ACKNOWLEDGEMENT

The authors used AI-assisted tools during manuscript drafting, code editing, experiment-log summarization, and figure-preparation support. The authors reviewed, edited, and verified the submitted content and remain responsible for the correctness, originality, and integrity of the paper.

REFERENCES

- [1] S. Srivastava, C. Li, M. Lingelbach, R. Martin-Martin, F. Xia, K. Vainio, Z. Lian, C. Gokmen, S. Buch, C. K. Liu, S. Savarese, H. Gweon, J. Wu, and L. Fei-Fei, “BEHAVIOR: Benchmark for everyday household activities in virtual, interactive, and ecological environments,” in *Proceedings of the Conference on Robot Learning*, 2022.
- [2] C. Li *et al.*, “BEHAVIOR-1K: A human-centered, embodied AI benchmark with 1,000 everyday activities and realistic simulation,” *arXiv preprint arXiv:2403.09227*, 2024.
- [3] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra, “Habitat: A platform for embodied AI research,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 9339–9347.
- [4] A. Szot, A. Clegg, E. Undersander, E. Wijmans, Y. Zhao, J. Turner, N. Maestre, M. Mukadam, D. S. Chaplot, O. Maksymets, A. Gokaslan, V. Vondrus, S. Dharur, F. Meier, W. Galuba, A. X. Chang, Z. Kira, V. Koltun, J. Malik, M. Savva, and D. Batra, “Habitat 2.0: Training home assistants to rearrange their habitat,” in *Advances in Neural Information Processing Systems*, 2021.

- [5] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, M. Deitke, K. Ehsani, D. Gordon, Y. Zhu, A. Gupta, and A. Farhadi, "AI2-THOR: An interactive 3d environment for visual AI," *arXiv preprint arXiv:1712.05474*, 2017.
- [6] X. Puig, K. Ra, M. Boben, J. Li, T. Wang, S. Fidler, and A. Torralba, "VirtualHome: Simulating household activities via programs," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8494–8502.
- [7] A. Padmakumar, J. Thomason, A. Shrivastava, P. Lange, A. Narayan-Chen, S. Gella, R. Piramuthu, G. Tur, and D. Hakkani-Tur, "TEACh: Task-driven embodied agents that chat," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.
- [8] A. Majumdar, A. Ajay, X. Zhang, P. Putta, S. Yenamandra, M. Henaff, S. Silwal, P. McVay, O. Maksymets, S. Arnaud, K. Yadav, Q. Li, B. Newman, M. Sharma, V. Berges, S. Zhang, P. Agrawal, Y. Bisk, D. Batra, M. Kalakrishnan, F. Meier, C. Paxton, A. Sax, and A. Rajeswaran, "OpenEQA: Embodied question answering in the era of foundation models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 16488–16498.
- [9] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han, R. Mottaghi, L. Zettlemoyer, and D. Fox, "ALFRED: A benchmark for interpreting grounded instructions for everyday tasks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020.
- [10] R. T. Snodgrass, *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999.
- [11] C. S. Jensen and R. T. Snodgrass, "Temporal data management," *IEEE Transactions on Knowledge and Data Engineering*, vol. 11, no. 1, pp. 36–44, 1999.
- [12] B. Babcock, S. Babu, M. Datar, R. Motwani, and J. Widom, "Models and issues in data stream systems," in *Proceedings of the ACM Symposium on Principles of Database Systems*, 2002, pp. 1–16.
- [13] A. Gupta and I. S. Mumick, "Maintenance of materialized views: Problems, techniques, and applications," *IEEE Data Engineering Bulletin*, vol. 18, no. 2, pp. 3–18, 1995.
- [14] Y. Zhuge, H. Garcia-Molina, J. Hammer, and J. Widom, "View maintenance in a warehousing environment," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1995, pp. 316–327.
- [15] C. C. Aggarwal and P. S. Yu, "A survey of uncertain data algorithms and applications," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 5, pp. 609–623, 2009.
- [16] T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance semirings," in *Proceedings of the ACM Symposium on Principles of Database Systems*, 2007, pp. 31–40.
- [17] J. Cheney, L. Chiticariu, and W.-C. Tan, "Provenance in databases: Why, how, and where," *Foundations and Trends in Databases*, vol. 1, no. 4, pp. 379–474, 2009.
- [18] O. Erling, A. Averbuch, J. Larriba-Pey, H. Chafi, A. Gubichev, A. Prat, M.-D. Pham, and P. Boncz, "The LDBC social network benchmark: Interactive workload," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 2022–2033, 2015.
- [19] A. Iosup, T. Hegeman, W. L. Ngai, S. Heldens, A. Prat-Pérez, T. Manhardt, H. Chafi, M. Capotă, N. Sundaram, M. Anderson, I. G. Tănase, Y. Xia, L. Nai, P. Boncz, and G. Siganos, "LDBC graphalytics: A benchmark for large-scale graph analysis on parallel and distributed platforms," *Proceedings of the VLDB Endowment*, vol. 9, no. 13, pp. 1317–1328, 2016.
- [20] A. Khelifati, F. Boukli Hacene, Y. Chabchoub, A. Dignös, J. Gamper, and G. Montoya, "TSM-Bench: Benchmarking time series database systems for monitoring applications," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3363–3376, 2023.
- [21] L. Qi, L. Chen, T. Weng, B. Hu, J. X. Yu, and X. Zhou, "LDBC finbench transaction workload," *Proceedings of the VLDB Endowment*, vol. 18, no. 11, pp. 3007–3020, 2025.
- [22] M. Malekpour, M. Riahi, M. Lamothe, and A. Mhedhbi, "SQLMorph: Query mutation and fine-grained metrics for text-to-SQL evaluation," in *Proceedings of the IEEE International Conference on Data Engineering*, 2026, experiment, Analysis, and Benchmark paper.
- [23] M. M. Najafi, X. Zhu, C. Kosyfaki, L. V. S. Lakshmanan, and R. Cheng, "BEACON: A benchmark for efficient and accurate counting of sub-graphs," *arXiv preprint arXiv:2504.10948*, 2025.
- [24] P. Mu, A. Chaves Carniel, A. Barbalace, and A. Shaikhha, "[experiment, analysis, and benchmark] systematic evaluation of plan-based adaptive query processing," *arXiv preprint arXiv:2511.16455*, 2025.
- [25] M. Garofalakis, J. Gehrke, and R. Rastogi, *Data Stream Management: Processing High-Speed Data Streams*. Springer, 2016.
- [26] N. N. Dalvi and D. Suciu, "Efficient query evaluation on probabilistic databases," in *Proceedings of the International Conference on Very Large Data Bases*, 2007, pp. 864–875.